

FC7xxx OSPI Integration Manual

Rev.0.1

Revision History

Revision	Date	Changes
0.1	2023/12/15	Initial release

Table of Contents

Chapter 1 Introduction	4
1.1 Introduction	4
Chapter 2 Building	5
2.1 Dependencies on Other Modules	5
2.2 Files Required for Compile	5
2.3 Add Plug-ins	6
Chapter 3 Memory	7
3.1 Sections in Memory Map	7
Chapter 4 Exclusive Area.....	9
Chapter 5 Interrupt Service Routine (ISR).....	101
Chapter 6 Error Report.....	112
6.1 Det.....	112
6.2 Dem.....	112
Chapter 7 Function Calls.....	123
7.1 Function Calls during Startup	123
7.2 Function Calls during Shutdown	123
7.3 Function Calls during Wake-up	123
7.4 Function Calls during Runtime	123
Chapter 8 Other Requirements	134
8.1 Notification, Callback, Callout	134
8.2 Macros	134
Chapter 9 Integration Steps.....	145

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the OSPI module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- MCU: This module provides the clock reference point for OSPI module.
- Port: This module provides pin port configuration relevant to OSPI buses.
- Common: This module is the basic module which used to choose the chip variant.
- DMA: This module provides the DMA channel reference point for OSPI module.
- Dem: This module is used for reporting production relevant error status.

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for enabling Development error detection.

Module initialization dependency

- Mcu: To use the OSPI module, the user needs to initialize MCU module first.
- Port: To use OSPI module, SCK, RWDS, CS, IO pins for each OSPI should be configured.
- DMA: To use OSPI module DMA function, the user needs to initialize DMA module first.

2.2 Files Required for Compile

OSPI module files:

- _MCAL/Src/Ospi/Src/CDD_Ospi.c
- _MCAL/Src/Ospi/Src/LLD_Ospi.c
- _MCAL/Src/Ospi/Src/CDD_Ospi_Irq.c
- _MCAL/Src/Ospi/include/CDD_Ospi.h
- _MCAL/Src/Ospi/include/LLD_Osp.h
- _MCAL/Src/Ospi/include/Ospi_Hw.h
- _MCAL/Src/Ospi/include/Ospi_MemMap.h
- _MCAL/Src/Ospi/include/Ospi_Reg.h
- _MCAL/Src/Ospi/include/Ospi_version.h
- _MCAL_generate/src/CDD_Ospi_PBcfg.c
- _MCAL_generate/src/CDD_Ospi_Cfg.c
- _MCAL_generate/include/CDD_Ospi_Cfg.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Std_Types.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Compiler.h

- `_MCAL/Src/Common/include/Compiler_Cfg.h`
- `_MCAL/Src/Common/include/CompilerDefinition.h`

Det module files:

- `Det.h`

Rte module files:

- `SchM_Ospi.h`

Dem module files:

- `Dem.h`

2.3 Add Plug-ins

OSPI module plug-ins are developed for EB tools, so, to use OSPI plug-ins on the EB tool, the user needs to add the OSPI module to the EB plug-ins folder first.

- 1) Copy the OSPI module(`_MCAL/EB_Plugins/eclipse/plugins/Ospi`) folder to EB tresos plug-ins (`EB/tresos/plugins/`) folder.
- 2) Set the OSPI module output location folder for the generated source file and header files.
- 3) Use EB tresos to configure the OSPI module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
OSPI_START_SEC_CONFIG_DATA_8 OSPI_STOP_SEC_CONFIG_DATA_8 OSPI_START_SEC_CONFIG_DATA_16 OSPI_STOP_SEC_CONFIG_DATA_16 OSPI_START_SEC_CONFIG_DATA_32 OSPI_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).
OSPI_START_SEC_CONFIG_DATA_UNSPECIFIED OSPI_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
OSPI_START_SEC_CONST_BOOLEAN OSPI_STOP_SEC_CONST_BOOLEAN OSPI_START_SEC_CONST_8 OSPI_STOP_SEC_CONST_8 OSPI_START_SEC_CONST_16 OSPI_STOP_SEC_CONST_16 OSPI_START_SEC_CONST_32 OSPI_STOP_SEC_CONST_32 OSPI_START_SEC_CONST_UNSPECIFIED OSPI_STOP_SEC_CONST_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
OSPI_START_SEC_CODE OSPI_STOP_SEC_CODE OSPI_START_SEC_RAMCODE OSPI_STOP_SEC_RAMCODE OSPI_START_SEC_CODE_AC OSPI_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
OSPI_START_SEC_VAR_NO_INIT_BOOLEAN OSPI_STOP_SEC_VAR_NO_INIT_BOOLEAN OSPI_START_SEC_VAR_NO_INIT_8 OSPI_STOP_SEC_VAR_NO_INIT_8 OSPI_START_SEC_VAR_NO_INIT_16 OSPI_STOP_SEC_VAR_NO_INIT_16 OSPI_START_SEC_VAR_NO_INIT_32 OSPI_STOP_SEC_VAR_NO_INIT_32 OSPI_START_SEC_VAR_NO_INIT_UNSPECIFIED OSPI_STOP_SEC_VAR_NO_INIT_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
OSPI_START_SEC_VAR_INIT_BOOLEAN OSPI_STOP_SEC_VAR_INIT_BOOLEAN OSPI_START_SEC_VAR_INIT_8 OSPI_STOP_SEC_VAR_INIT_8	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by

Section Name	Section Type	Description
OSPI_START_SEC_VAR_INIT_16		startup code (data).
OSPI_STOP_SEC_VAR_INIT_16		
OSPI_START_SEC_VAR_INIT_32		
OSPI_STOP_SEC_VAR_INIT_32		
OSPI_START_SEC_VAR_INIT_UNSPECIFIED		
OSPI_STOP_SEC_VAR_INIT_UNSPECIFIED		

Chapter 4 Exclusive Area

OSPI module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the Ospi driver:

- LLD_Ospi.c:
 - OSPI_LLD_UpdateLUT: exclusive area 0

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
OSPI0	OSPI_IRQHandler	73

Chapter 6 Error Report

6.1 Det

Function Name	Error Type
Ospi_Init	OSPI_E_ALREADY_INITIALIZED; OSPI_E_PARAM_POINTER
Ospi_DeInit	OSPI_E_UNINIT
Ospi_SetFlashConfig	OSPI_E_UNINIT;
Ospi_UpdateLUT	OSPI_E_UNINIT; OSPI_E_PARAM_POINTER; OSPI_E_INVALID_CHANNEL
Ospi_HyperBusSyncCommandRead	OSPI_E_UNINIT; OSPI_E_PARAM_POINTER; OSPI_E_PARAM_OUTRANGE
Ospi_HyperBusSyncCommandWrite	OSPI_E_UNINIT; OSPI_E_PARAM_POINTER; OSPI_E_PARAM_OUTRANGE
Ospi_GetStatus	OSPI_E_UNINIT;
Ospi_HyperBusSyncCommandDmaInit	OSPI_E_UNINIT; OSPI_E_PARAM_POINTER; OSPI_E_PARAM_OUTRANGE
Ospi_HyperBusSyncCommandEnableTxDma	OSPI_E_UNINIT
Ospi_HyperBusSyncCommandEnableRxDma	OSPI_E_UNINIT
Ospi_HyperBusSyncCommandSetTxFifoWatermark	OSPI_E_UNINIT
Ospi_HyperBusSyncCommandSetRxFifoWatermark	OSPI_E_UNINIT
Ospi_HyperBusSyncCommandDmaRead	OSPI_E_UNINIT
Ospi_HyperBusSyncCommandDmaWrite	OSPI_E_UNINIT

6.2 Dem

None

Chapter 7 Function Calls

7.1 Function Calls during Startup

The API needs be called is `Ospi_Init(const Ospi_ConfigType *pConfigPtr);`

7.2 Function Calls during Shutdown

None

7.3 Function Calls during Wake-up

None

7.4 Function Calls during Runtime

None

Chapter 8 Other Requirements

8.1 Notification, Callback, Callout

- Notification

If the user configures notifications and the value is not "NULL_PTR" or "NULL", the user shall implement the notification function in the user code.

- Callback

None

- Callout

None

8.2 Macros

Please check various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:

AUTOSAR_OS_NOT_USED need defined.

In this case, If USE_SW_VECTOR_MODE is not defined, the user needs to map the interrupt vector table function to ISR function, for example:

```
extern ISR(Ospi_IsrTransferError);

void OSPI_IRQHandler(void)
{
    Ospi_IsrTransferError();
}
```

- If OS is used:

Do not define AUTOSAR_OS_NOT_USED.

Chapter 9 Integration Steps

- 1) Configure the OSPI module and generate configuration files (please refer to Building chapter for details).
- 2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
- 3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
- 4) Build the OSPI module with dependent modules.